

```

record("Initial")

while buffer or len(stack) > 1:

    if len(stack) >= 2 and stack[-1] != "ROOT" and stack[-2] != "ROOT":

        top = doc[stack[-1]]
        second = doc[stack[-2]]

        if second.head == top:
            stack.pop(-2)
            record("LEFT-ARC", f"{top.text} → {second.text}")
            continue

        if top.head == second:
            stack.pop()
            record("RIGHT-ARC", f"{second.text} → {top.text}")
            continue

    if buffer:
        stack.append(buffer.popleft())
        record("SHIFT")
    else:
        break

print(f"{'STACK':<40} {'BUFFER':<60} {'TRANSITION':<15} DEPENDENCY")
print("-" * 110)

for stack_s, buffer_s, action, dep in steps:
    print(f"{'str(stack_s):<40'} {'str(buffer_s):<60'} {'action:<15'} {'dep:}'")

```

STACK	BUFFER	TRANSITION	DEPENDENCY
['ROOT']	['I', 'parsed', 'this', 'sentence', 'correctly']	Initial	
['ROOT', 'I']	['parsed', 'this', 'sentence', 'correctly']	SHIFT	
['ROOT', 'I', 'parsed']	['this', 'sentence', 'correctly']	SHIFT	
['ROOT', 'parsed']	['this', 'sentence', 'correctly']	LEFT-ARC	parsed → I
['ROOT', 'parsed', 'this']	['sentence', 'correctly']	SHIFT	
['ROOT', 'parsed', 'this', 'sentence']	['correctly']	SHIFT	
['ROOT', 'parsed', 'sentence']	['correctly']	LEFT-ARC	sentence → this
['ROOT', 'parsed']	['correctly']	RIGHT-ARC	parsed → sentence
['ROOT', 'parsed', 'correctly']	[]	SHIFT	
['ROOT', 'parsed']	[]	RIGHT-ARC	parsed → correctly

Implement a trans. based dependency parser using spaCy that process a sentence by applying **SHIFT**, **LEFT-ARC**, and **RIGHT-ARC** operations while recording the stack buffer and dependency relation at each step.

```

1s
import spacy
from collections import deque

nlp = spacy.load("en_core_web_sm")

# sentence = "Students study NLP concepts"
sentence="The dog chased the cat"
doc = nlp(sentence)

stack = ["ROOT"]
buffer = deque(range(len(doc)))
steps = []

def record(action, dep=""):
    steps.append((
        [doc[i].text if i != "ROOT" else "ROOT" for i in stack],
        [doc[i].text for i in buffer],
        action,
        dep
    ))

record("Initial")

while buffer or len(stack) > 1:

    if len(stack) >= 2 and stack[-1] != "ROOT" and stack[-2] != "ROOT":

        top = doc[stack[-1]]
        second = doc[stack[-2]]

        if second.head == top:
            stack.pop(-2)
            record("LEFT-ARC", f"{top.text} → {second.text}")
            continue

        if top.head == second:
            stack.pop()
            record("RIGHT-ARC", f"{second.text} → {top.text}")
            continue

    if buffer:
        table = 

|  | STACK                            | BUFFER                                 | TRANSITION | DEPENDENCY   |
|--|----------------------------------|----------------------------------------|------------|--------------|
|  | ['ROOT']                         | ['The', 'dog', 'chased', 'the', 'cat'] | Initial    |              |
|  | ['ROOT', 'the']                  | ['dog', 'chased', 'the', 'cat']        | SHIFT      |              |
|  | ['ROOT', 'The', 'dog']           | ['chased', 'the', 'cat']               | SHIFT      |              |
|  | ['ROOT', 'dog']                  | ['chased', 'the', 'cat']               | LEFT-ARC   | dog - The    |
|  | ['ROOT', 'dog', 'chased']        | ['the', 'cat']                         | SHIFT      |              |
|  | ['ROOT', 'chased']               | ['the', 'cat']                         | LEFT-ARC   | chased - dog |
|  | ['ROOT', 'chased', 'the']        | ['cat']                                | SHIFT      |              |
|  | ['ROOT', 'chased', 'the', 'cat'] | []                                     | SHIFT      |              |
|  | ['ROOT', 'chased', 'cat']        | []                                     | LEFT-ARC   | cat - the    |
|  | ['ROOT', 'chased']               | []                                     | RIGHT-ARC  | chased - cat |


        else:
            break

    print(f"{'STACK':<40} {'BUFFER':<60} {'TRANSITION':<15} DEPENDENCY")
    print("-" * 110)

    for stack_s, buffer_s, action, dep in steps:
        print(f"{'str(stack_s)':<40} {'str(buffer_s)':<60} {action:<15} {dep}")

```

STACK	BUFFER	TRANSITION	DEPENDENCY
['ROOT']	['The', 'dog', 'chased', 'the', 'cat']	Initial	
['ROOT', 'The']	['dog', 'chased', 'the', 'cat']	SHIFT	
['ROOT', 'The', 'dog']	['chased', 'the', 'cat']	SHIFT	
['ROOT', 'dog']	['chased', 'the', 'cat']	LEFT-ARC	dog → The
['ROOT', 'dog', 'chased']	['the', 'cat']	SHIFT	
['ROOT', 'chased']	['the', 'cat']	LEFT-ARC	chased → dog
['ROOT', 'chased', 'the']	['cat']	SHIFT	
['ROOT', 'chased', 'the', 'cat']	[]	SHIFT	
['ROOT', 'chased', 'cat']	[]	LEFT-ARC	cat → the
['ROOT', 'chased']	[]	RIGHT-ARC	chased → cat

Run a tran.-based dep. parser using spacy on the sentence "The dog chased the cat" and record each SHIFT, LEFT-ARC and RIGHT-ARC step with the stack, buffer and dependency relations.

Dependency Tree Parser

```

!pip install -q spacy
!python -m spacy download en_core_web_sm

Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none-any.whl (12.8 MB)
    12.8/12.8 MB 99.2 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
▲ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.

```

1
1s



```
import spacy
from spacy import displacy

# Load English model
nlp = spacy.load("en_core_web_sm")

sentence = "I parsed this sentence correctly"

doc = nlp(sentence)

print("Dependency Relations:\n")

# Print dependency tree relations
for token in doc:

    if token.dep_ == "ROOT":
        print(f"ROOT → {token.text}")

    else:
        print(f"{token.head.text} → {token.text}")

# Build custom dependency tree dictionary
tree = {}

for token in doc:

    head = token.head.text

    if token.dep_ == "ROOT":
        head = "ROOT"

    if head not in tree:
        tree[head] = []

    tree[head].append(token.text)

print("\nCustom Dependency Tree:\n")

for head in tree:
    for child in tree[head]:
        print(f"{head} → {child}")

# Visual tree
print("\nVisual Dependency Tree:\n")
displacy.render(doc, style="dep", jupyter=True)
```

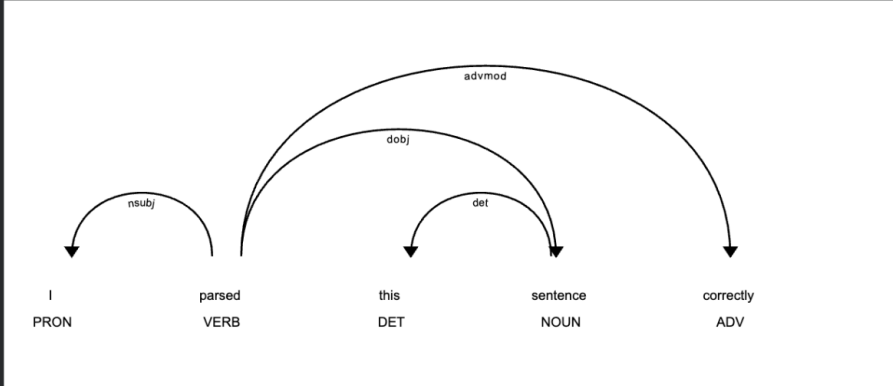
Dependency Relations:

```
parsed → I  
ROOT → parsed  
sentence → this  
parsed → sentence  
parsed → correctly
```

Custom Dependency Tree:

```
parsed → I  
parsed → sentence  
parsed → correctly  
ROOT → parsed  
sentence → this
```

Visual Dependency Tree:



Use spacy to perform dependency parsing on the sentence and display the grammatical relationship between words along with a visual dependency tree.

Parse Tree

```
!pip install -q spacy graphviz
!python -m spacy download en_core_web_sm

... Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none-any.whl (12.8 MB)
    12.8/12.8 MB 99.8 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in order to load all the package's dependencies. You can do this by selecting the 'Restart kernel' or 'Restart runtime' option.
```

```
import spacy
from graphviz import Digraph

# Load spaCy model
nlp = spacy.load("en_core_web_sm")

sentence = "I parsed this sentence correctly"
doc = nlp(sentence)

# Create graph
tree = Digraph()

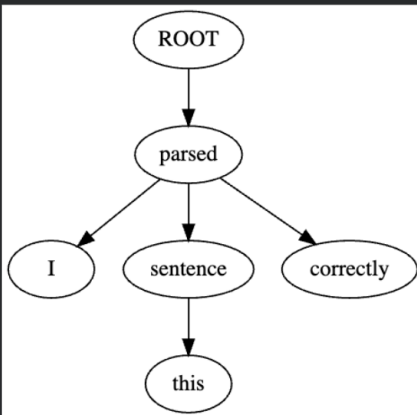
# Add ROOT node
tree.node("ROOT")

# Add nodes and edges
for token in doc:
    tree.node(token.text)

    if token.dep_ == "ROOT":
        tree.edge("ROOT", token.text)

    else:
        tree.edge(token.head.text, token.text)

# Display tree
tree
```



→ use spacy to parse the sentence and Graphviz to construct and visualize the dep. parse tree showing relationships between words.

Q1-B) A sentence containing n words require $2n$ parsing steps. This happens because every word must first be shifted from the buffer of the stack (n shift operation) and each word also forms one dependency relation using an ARC operation (Left ARC or Right ARC). Therefore, the total no. of transitions is $n + n = 2n$.

Q2-A)

```
1 28s ▶ import torch
import torch.nn as nn
import matplotlib.pyplot as plt
import numpy as np

# -----
# LSTM Models
# -----
class BasicLSTM(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers):
        super().__init__()
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, 1)

    def forward(self, x):
        out, _ = self.lstm(x)
        return self.fc(out[:, -1, :])

class OptimizedLSTM(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers):
        super().__init__()
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers,
                              batch_first=True, dropout=0.3)
        self.fc = nn.Linear(hidden_size, 1)

    def forward(self, x):
        out, _ = self.lstm(x)
        return self.fc(out[:, -1, :])

# -----
# Generate Time Series
# -----
time = np.linspace(0, 100, 500)
series = np.sin(time)

seq_len = 10
X = []
y = []

for i in range(len(series) - seq_len):
    X.append(series[i:i + seq_len])
    y.append(series[i + seq_len])

X = torch.tensor(X).float().unsqueeze(-1)
y = torch.tensor(y).float().unsqueeze(1)
```



```

# -----
# Models
# -----
basic_model = BasicLSTM(1,50,2)
opt_model = OptimizedLSTM(1,128,3)

criterion = nn.MSELoss()

opt1 = torch.optim.Adam(basic_model.parameters(),lr=0.01)
opt2 = torch.optim.Adam(opt_model.parameters(),lr=0.01)

epochs = 40
basic_loss=[]
opt_loss=[]

for e in range(epochs):

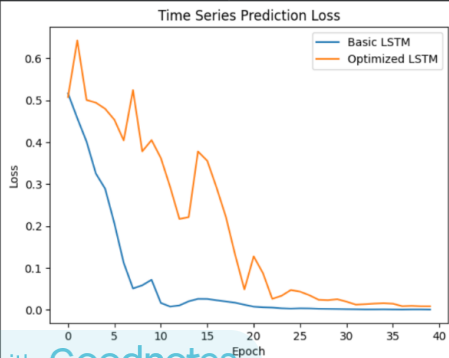
    opt1.zero_grad()
    out = basic_model(X)
    loss = criterion(out,y)
    loss.backward()
    opt1.step()
    basic_loss.append(loss.item())

    opt2.zero_grad()
    out = opt_model(X)
    loss = criterion(out,y)
    loss.backward()
    opt2.step()
    opt_loss.append(loss.item())

# Graph
plt.plot(basic_loss,label="Basic LSTM")
plt.plot(opt_loss,label="Optimized LSTM")
plt.title("Time Series Prediction Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()

```

... /tmp/ipykernel_515/3263646004.py:46: UserWarning: Creating a tensor from a list of numpy.ndarrays is extremely slow. Please consider converting to a torch.tensor first.
X = torch.tensor(X).float().unsqueeze(-1)



→ Implement Basic and optimized LSTM model using Pytorch to predict a sine wave time-series dataset and compares their training performance by plotting the loss over epochs.